WEB PROGRAMLAMA I · MDN ÇEKİRDEK MODÜLLER

HTML ile İçerik Yapılandırma

index.html

```
<!doctype html>
<html lang="tr">
  <head>
    <title>Elit Backend</title>
  </head>
  <body>
    <h1>Merhaba, web!</h1>
  </body>
</html>
```

Structuring content with HTML

Elit Backend | Çekirdek Modüller, ilk başlık

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs, Vue.js, Three.js vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.

GİRİŞ

Bugünün haritası

<html> 1

Sözdizimi ve head

öğe, öznitelik, belge iskeleti, metadata

<p> 2

Metin

başlıklar, vurgu, listeler, gelişmiş metin

<header> 3

Yapı ve bağlantılar

belge yapısı ve linkler

<img> 4

Medya

resim, video ve ses

<table> 5

Tablolar

temeller ve erişilebilirlik

<form> 6

Formlar ve debug

form, düğme, HTML hata ayıklama

1989 → bugün

HTML'in kısa tarihi

1989

Tim Berners-Lee, CERN'de bilgiyi linklerle paylaşan bir sistem önerir: web doğar.

1991

"HTML Tags" belgesi yayımlanır: ilk 18 etiket (p, a, h1… gibi).

1995

HTML 2.0: ilk resmî standart. Doctype'lar bu dönemde kural seti gösterirdi.

1997–99

W3C, HTML 3.2, 4.0 ve 4.01'i yayımlar; tablolar, formlar, stil desteği gelişir.

2000

XHTML 1.0: HTML'i XML'in katı kurallarıyla yazma denemesi.

2004

Apple, Mozilla ve Opera WHATWG'yi kurar; yeni nesil HTML üzerinde çalışır.

2014

HTML5 W3C standardı olur: video, audio, header, nav, main, article…

2019

HTML Living Standard: sürüm numarası yok, sürekli güncellenen tek standart.

Benzetme — HTML basılıp biten bir kitap değil, yaşayan bir sözlük gibi: yeni kelimeler eklenir, eskiler emekliye ayrılır.

Kaynak: W3C · WHATWG · MDN

SÖZDİZİMİ

Bir HTML öğesinin anatomisi

ogeler.html

```
<p>My cat is very grumpy</p>
```

açılış etiketi	içerik	kapanış etiketi

HTML, bir işaretleme dilidir: tarayıcıya sayfayı nasıl yapılandıracağını söyler. Kapanış etiketini unutmak yaygın bir başlangıç hatasıdır ve tuhaf sonuçlar üretir.

İç içe geçme (nesting)

✔ `<p>…<strong>very</strong>…</p>`
✖ `<p>…<strong>very</p></strong>`

Etiketler birbirinin ya tamamen içinde ya tamamen dışında olmalı.

Boş (void) öğeler

Tek etiketten oluşur, içine başka içerik alamaz. Örnek:
 satır sonu ekler.
 yazımı da geçerli ama kapanış eğik çizgisi şart değil.

Benzetme	matruşka bebekler gibi: küçük bebek büyüğünün içinde tamamen durmalı, yarısı dışarıda kalmamalı.

`class="…"`

Öznitelikler (attributes)

resim.html

```
<img src="firefox.png"
     alt="Firefox simgesi"
     width="300" />
```

src: resmin adresi (zorunlu).

alt: göremeyenler için metin açıklaması. Anlam taşıyan her resimde olmalı.

width, height: piksel cinsinden boyut.

Boolean öznitelikler

Değersiz yazılır: <input disabled />. Varsa true, yoksa false olur (değeri ne olursa olsun).

Tırnak kuralı

✗ `title=The Mozilla homepage`
✓ `title="The Mozilla homepage"`

Tırnaksız yazarsan tarayıcı üç öznitelik sanar. Her zaman tırnak kullan.

Benzetme — ürün etiketi gibi: ürünün kendisi (içerik) değil, hakkındaki ek bilgiler (beden, renk, adres).

Bir HTML belgesinin iskeleti

index.html

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <title>My test page</title>
  </head>
  <body>
    <p>This is my page</p>
  </body>
</html>
```

1. **`doctype`** — tarihi bir kalıntı ama her sayfanın en üstünde şart
2. **`html`** — kök öğe, sayfanın tüm içeriğini sarar
3. **`head`** — sayfada görünmeyen bilgiler için kap
4. **`meta charset`** — karakter kodlaması: utf-8
5. **`title`** — sekmede ve yer imlerinde görünen başlık
6. **`body`** — ekranda görünen her şey burada

Benzetme	kitap gibi: kapak her şeyi sarar (html), künye sayfası (head), okunan sayfalar (body).

`<  &  <!-- -->`

Boşluk, özel karakterler ve yorumlar

Boşluk

Çoğu zaman isteğe bağlıdır, okunabilirlik içindir. Tarayıcı ardışık boşlukları tek boşluğa indirir (<pre> gibi istisnalar var). MDN'de her iç içe öğe 2 boşluk girintilenir.

Özel karakterler

Karakter	Yazılışı
<	<
>	>
"	"
'	'
&	&

& ile başlar, ; ile biter.

Yorumlar

yorum.html

```
<!-- <p>Ben!</p> -->
```

<!-- ile başlar, --> ile biter. Tarayıcı yok sayar, kullanıcı görmez. Kendine ve ekibe not bırakmak içindir.

Benzetme yorum, kitabın kenarına yazılan kurşun kalem notu gibi: yazan bilir, okuyucu (tarayıcı) görmez.

METADATA

head: sayfanın künyesi

<title> ve <h1> aynı şey değil

<h1> sayfada görünür, genelde sayfa başına bir kez kullanılır. <title> ise belgenin başlığıdır: sekmede, yer imlerinde ve arama sonuçlarında kullanılır.

charset: <meta charset="utf-8" />

utf-8 hemen her dilin karakterini içerir. ISO-8859-1 seçilirse Japonca gibi metinler bozulur. Her sayfada ayarla.

description ve author

name ve content özniteliği: <meta name="description" content="..."> . Açıklama arama sonucunda <title> ile birlikte görünür ve SEO için faydalıdır.

keywords ve Open Graph

keywords arama motorları tarafından yok sayılır (spam yüzünden). Open Graph (og:image, og:title) link paylaşımında resim ve açıklama gösterir.

Benzetme	kitabın künyesi gibi: yazar, konu, özet. Okuyucu sayfalarda görmez ama kütüphane (arama motoru) kullanır.

`<link>` `<script>`

Simge, stil, script ve dil

Favicon

```
head
<link rel="icon" href="/favicon.ico" />
```

Sekmede ve yer imlerinde görünen 16 piksellik simge. Yolun başındaki / sitenin kök dizinini gösterir.

CSS bağlantısı

```
head
<link rel="stylesheet" href="my-css-file.css" />
```

Her zaman head içine gider. rel ve href öznitelikleri gerekir.

JavaScript

```
head
<script src="my-js-file.js" defer></script>
```

defer: HTML ayrıştırıldıktan sonra çalış. Henüz olmayan öğeye erişme hatasını önler. Kapanış etiketi gerekir.

Sayfa dili

```
head
<html lang="en-US">
```

Arama motorları ve ekran okuyucular için önemli. 'six' kelimesi Fransızca ve İngilizcede farklı okunur.

Benzetme	evin kapısı gibi: plaket (favicon), duvar rengi (CSS), elektrik (JS), tabelanın dili (lang).

<h1> <h2> <p>

Başlıklar ve paragraflar

hikaye.html

```
<h1>The Crushing Bore</h1>

<p>By Chris Mills</p>

<h2>Chapter 1: The dark night</h2>
<p>It was a dark night…</p>

<h2>Chapter 2: The eternal silence</h2>
<h3>The specter speaks</h3>
<p>Several more hours had passed…</p>
```

Sayfada tek <h1>

En üst seviye başlık budur, diğerleri onun altında yer alır.

Sırayı bozma

Alt başlık için h3, onun altı için h2 kullanmak gibi ters sıralama yapma; seviyeler sırayla gitmeli.

En fazla 3 seviye

Çok derin hiyerarşi dolaşılmaz olur. Gerekirse içeriği birkaç sayfaya böl.

Benzetme kitabın içindekiler sayfası gibi: kitap adı (h1), bölümler (h2), alt bölümler (h3). Sıra karışırsa içindekiler anlamsız olur.

<h1> vs

Yapı ve semantik neden önemli?

Başlık yoksa ne olur?

Kullanıcılar: sayfaya hızlı göz atar, çoğu zaman sadece başlıkları okur.

Arama motorları: başlık içeriğini önemli anahtar kelime sayar (SEO).

Ekran okuyucular: başlıklardan belgenin ana hatlarını çıkarır.

CSS ve JavaScript: hedef alabilmek için içeriği saran öğeler gerekir.

Semantik: doğru öğe, doğru anlam

✓ Anlamlı

dogru.html

```
<h1>This is a top level heading</h1>
```

✕ Sadece görünüşü başlık

yanlis.html

```
<span style="font-size: 32px;
  display: block;">
  Is this a heading?</span>
```

span'in anlamı yoktur: büyük yazılsa da başlık avantajlarının hiçbirini almaz.

Benzetme trafik ışığı gibi: kırmızı herkes için “dur” demektir. h1 de herkes için “ana başlık” demektir, büyük yazmak yetmez.

Vurgu ve önem

METIN

: vurgu

vurgu.html

```
<p>I am <em>glad</em> you weren't
   <em>late</em>.</p>
```

Cümlenin anlamını değiştirir. Ekran okuyucu tonu değiştirebilir.

: güçlü önem

onem.html

```
<p>This liquid is
   <strong>highly toxic</strong>.</p>
```

Kritik ifadeleri işaretler. İç içe yazılabilir.

<b>, <i>, <u> ne olacak?

em ve strong: sadece italik veya kalın görünmek için kullanılmaz; o iş span ve CSS'indir.

Eski sunumsal öğeler (<big>, <font>) artık kullanılmaz.

HTML5 b, i, u'yu yeni anlamlarla yeniden tanımladı.

Kural: daha uygun öğe yoksa kullan; strong, em, mark, span önce düşünülmeli.

i: yabancı sözcük, tür adı, teknik terim, düşünce.

b: anahtar sözcük, ürün adı, giriş cümlesi.

u: özel isim, yazım hatası. Altı çizili yazı link sanılır.

Benzetme konuşmada sesini yükseltmek (em) ile “dikkat, tehlikeli!” demek (strong) farklıdır: biri anlamı değiştirir, öbürü uyarır.

  <dl>

Listeler: üç tür

 Sırasız liste

liste.html

```
<ul>
  <li>milk</li>
  <li>eggs</li>
</ul>
```

Sırası önemli olmayan öğeler: alışveriş listesi gibi.

 Sıralı liste

liste.html

```
<ol>
  <li>Turn right</li>
  <li>Turn left</li>
</ol>
```

Sırası önemli öğeler: yol tarifi, tarif adımları gibi.

<dl> Tanım listesi

liste.html

```
<dl>
  <dt>aside</dt>
  <dd>In drama, …</dd>
</dl>
```

Terim (dt) ve açıklama (dd) çiftleri. Bir terimin birden çok açıklaması olabilir.

İç içe liste: alt liste, üst listenin <li> öğesinin içine yazılır. Navigasyon menüleri de çoğunlukla liste ile yapılır.

Benzetme	market listesi (ul) sırasızdır, yemek tarifi (ol) karışırsa yemek olmaz, sözlük (dl) kelimeyi anlamıyla eşler.

Alıntı, kısaltma ve adres

Alıntılar

alinti.html

```
<blockquote cite="https://…">
  <p>uzun alıntı</p>
</blockquote>
<q cite="https://…">kısa alıntı</q>
```

blockquote: blok alıntı. q: satır içi alıntı (tırnağı kendisi koyar).

cite özniteliği: tarayıcılar pek kullanmaz. Kaynağı metinde link ile ver.

<cite> öğesi: kaynağın başlığı (kitap adı gibi), varsayılan italik.

<abbr>: kısaltma

abbr.html

```
<abbr title="Reverend">Rev.</abbr>
```

İlk kullanımda açılımı metinde ver. acronym kaldırıldı, hepsi için abbr.

<address>: iletişim

En yakın article veya body için iletişim bilgisi içindir: sitenin footer'ı ya da makalenin yazarı. Alakasız adres listesi için değil.

Benzetme	kenara alıntı yazmak gibi: uzun alıntıyı ayrı paragrafla (blockquote), kısa alıntıyı tırnakla (q), kaynağı dipnotla (cite).

`<code>` `<sup>` `<time>`

Kod, simge ve tarih öğeleri

Bilgisayar kodu

code: genel kod parçası.

pre: boşlukları korur (kod blokları).

var: değişken adı.

kbd: klavye girdisi.

samp: program çıktısı.

kbd.html

```
<kbd>Ctrl</kbd> +
<kbd>A</kbd>
```

Üst ve alt simge

Tarihlerde, kimyasal formüllerde ve matematik denklemlerinde anlamı doğru vermek için kullanılır.

simge.html

```
25<sup>th</sup> of May

C<sub>8</sub>H<sub>10</sub>
N<sub>4</sub>O<sub>2</sub>

x<sup>2</sup> is 9
```

`<time>`: tarih ve saat

İnsanlar tarihi çok farklı yazar (20/01/16, 01/20/16, 20 Ocak). Bilgisayar bunları kolay tanıyamaz. datetime özniteliği kesin ve makine tarafından okunur biçim verir.

time.html

```
<time datetime="2016-01-20">
  20 January 2016</time>
```

Benzetme tarihi “öbür ay” diye yazmak ile takvime girmek farklıdır: time, insan yazısını bilgisayarın okuyacağı tarihe çevirir.

Belge yapısı: sayfanın bölgeleri

index.html

```
<body>
  <header><h1>Site</h1></header>
  <nav><ul>…</ul></nav>
  <main>
    <article>
      <h2>Makale</h2>
      <section>…</section>
    </article>
    <aside>İlgili linkler</aside>
  </main>
  <footer>&copy; 2026</footer>
</body>
```

<header>	giriş: logo, başlık. body içindeyse tüm sitenin, article içindeyse o bölümün başlığı
<nav>	ana gezinme linkleri (ikincil linkler buraya girmez)
<main>	sayfaya özgü içerik; sayfada bir kez, doğrudan body içinde
<article>	kendi başına anlamlı blok: blog yazısı, haber
<section>	tek bir tema/işlev; başlıkla başlamalı
<aside>	dolaylı ilgili içerik: yazar biyografisi, ilgili linkler
<footer>	son içerik: telif, iletişim, küçük yazılar

Benzetme	bir gazete gibi: künye (header), içindekiler (nav), haberler (main/article), köşe yazısı kutusu (aside), alt bilgi (footer).

`<div>` `<span>` `<br>` `<hr>`

Anlamsız sarmalayıcılar ve site planlama

`<div>` ve `<span>`

Semantiği yoktur; sadece CSS/JS ile hedeflemek için sararlar. span satır içi, div blok düzeyidir. Uygun semantik öğe yoksa kullan, mutlaka class ver. Aşırı div kodu karmaşıklaştırır.

`<br>` ve `<hr>`

br: blok içinde satır sonu (adres, şiir). hr: konu ya da sahne değişimi (tematik ara), yatay çizgi olarak görünür. İkisi de boş (void) öğedir.

Bilgi mimarisi: 5 adımda site planı

1. Her sayfada ortak olanları belirle (header, menü, footer).
2. Tipik sayfa düzenini kâğıda çiz.
3. Sitede olacak tüm içeriği listele (beyin fırtınası).
4. İçerikleri gruplara ayır (kart sıralama).
5. Site haritası çiz: sayfalar kutu, kullanıcı akışı çizgi.

Benzetme — div, üzerinde hiçbir şey yazmayan karton koli gibi: işe yarar ama içinde ne olduğunu kimse bilmez.

`<a href="…">`

Bağlantılar: web'i web yapan şey

link.html

```
<a href="https://www.mozilla.org/"
   title="Mozilla hakkında">
  Mozilla ana sayfası</a>
```

href: hedef adres (URL).

title: fareyle üzerine gelince ipucu. Klavye ve dokunmatik kullanıcılar göremez; önemli bilgiyi metne yaz.

Blok ve resim linkleri

Başlık gibi blok öğeler de <a> ile sarılabilir. Resim linkinde img'nin alt metni linkin adı olur, anlamlı yaz.

İyi link metni

✔ `<a>Firefox'u indir</a>`
✘ `<a>Buraya tıkla</a>`

Ekran okuyucular linkleri bağlamdan kopuk okur. Kısa, açıklayıcı yaz; “link” kelimesini ve çıplak URL'yi yazma.

Benzetme yol tabelası gibi: “buraya” yazan tabela işe yaramaz, “Kastamonu 12 km” yazan işe yarar.

../ / #id

URL yolları, parçalar ve özel linkler

Yol türü	Örnek	Anlamı
Aynı klasör	`contacts.html`	bulunduğun dosyaya göre
Alt klasör	`projects/index.html`	klasörün içine in
Üst klasör	`../pdfs/brief.pdf`	bir üst klasöre çık, sonra in
Kök göreli	`/pdfs/brief.pdf`	sitenin kökünden başla
Tam URL	`https://site.com/x`	web'in her yerinden aynı yer

Site içi linklerde göreli yol tercih et: alan adı değişse de çalışır. Dış linkte tam URL şart.

Özel durumlar

#id: sayfadaki belirli bir başlığa atla (contacts.html#adres).

download: dosyayı açmak yerine indir; dosya türü ve boyutunu metinde belirt.

target="_blank": yeni sekme; kullanıcıyı şaşırtabilir, bilinçli kullan.

mailto: e-posta istemcisini açar; ?subject= ve &body= eklenebilir.

Benzetme adres tarifi gibi: “yan daire” (göreli), “bir kat yukarı” (../), “Atatürk Cad. No:5” (tam URL).

`<img>`

Resimler: src, alt, width, height

resim.html

```
<img src="images/dinosaur.jpg"
     alt="Dinozor iskeletinin baş ve gövdesi"
     width="400" height="341" />
```

Void ve replaced öğe: içeriği ve boyutu dış dosyadan gelir.

src: göreli yol tercih et, açıklayıcı dosya adı ver (img835.png değil). Başkasının sunucusundan izinsiz bağlama (hotlink).

width/height: piksel, birimsiz. Yer ayırır, sayfa kaymasını (layout shift) önler. Yeniden boyutlandırma için CSS kullan.

alt metni: resmin görevine göre

Görev	Ne yazılır?
Süs	CSS arka planı ya da alt=""
İçerik	önemli bilgiyi kısaca ver
Link	linkin nereye gittiğini
Metinli resim	kaçın; gerekirse metni alt'a yaz

alt; ekran okuyucu, kırık yol, yavaş bağlantı ve arama motoru için hayat kurtarır.

Benzetme	alt metni, radyoda maç anlatan spiker gibi: görmeyene sahnede ne olduğunu anlatır.

`<figure>` `<figcaption>`

Resim yazısı ve CSS arka planı

figure.html

```
<figure>
  <img src="dinosaur.jpg" alt="…"
       width="400" height="341" />
  <figcaption>
    Manchester Müzesi'ndeki T-Rex
  </figcaption>
</figure>
```

div + p ile görsel olarak aynı görünür ama resim ile yazı arasında anlamsal bağ kurulmaz. figcaption bu bağı kurar. figure sadece resim değil; kod, tablo, video da içerebilir. Altyazı herkes içindir, alt ise resmin yerine geçer: aynı şeyi yazmasınlar. title özniteliği önerilmez, bilgiyi figcaption'a taşı.

`<img>` mi, CSS mi?

	`<img>`	background-image
Amaç	anlamlı içerik	sadece süs
Semantik	alt ile metin karşılığı	ekran okuyucu görmez
Konumlama	daha az esnek	daha kolay

Lisans: lisans belirtilmemişse tüm hakları saklı say. CC, MIT gibi izinli lisanslarda atıf şartlarına uy.

Benzetme müzedeki eser ve altındaki küçük plaket gibi: figure vitrin, figcaption plaket.

`<video>` `<source>`

Video: çoklu kaynak ve öznitelikler

video.html

```
<video controls poster="poster.png">
  <source src="rabbit.mp4"
          type="video/mp4" />
  <source src="rabbit.webm"
          type="video/webm" />
  <p>Tarayıcın desteklemiyor.
    <a href="rabbit.mp4">İndir</a></p>
</video>
```

Tarayıcı desteklediği ilk source'u oynatır. type (MIME türü) yazılırsa desteklemediği dosyayı indirmeden atlar. İçerideki p, desteklemeyen tarayıcı için yedek içeriktir.

`controls`	oynat/durdur/ses arayüzü. Kullanıcı kontrolü şart (epilepsi riski).
`autoplay`	otomatik başlat; rahatsız edici, önerilmez.
`loop / muted`	bitince başa dön / sessiz başla.
`poster`	oynatılmadan önce görünen kapak resmi.
`preload`	none, metadata ya da auto: ne kadar önden yüklensin.
`width/height`	boyut; en-boy oranı korunur.

Benzetme — çok dilli menü gibi: garson (tarayıcı) okuyabildiği ilk dili seçer; hiçbirini bilmiyorsa yedek kartı (p) uzatır.

`<audio>` `<track>`

Ses, kapsayıcı/kodek ve altyazı

`<audio>`

ses.html

```
<audio controls>
  <source src="viper.mp3"
   type="audio/mp3" />
</audio>
```

video ile aynı mantık. Görsel kısmı olmadığı için width, height ve poster desteklemez.

Kapsayıcı ve kodek

Kapsayıcı: MP4, WebM, Ogg. Ses, görüntü ve altyazı izlerini paketler.

Kodek: veriyi sıkıştırıp açar (H.264, VP9, AAC, Opus).

Pratik: MP4 + WebM vermek çoğu tarayıcıyı kapsar.

`<track>` ve WebVTT

altyazi.vtt

```
WEBVTT
00:00:22.230 -->
00:00:24.606
İlk altyazı.
```

kind: subtitles, captions, descriptions. src, srclang, label ver. source'lardan sonra yazılır. İşitme engelliler ve SEO için faydalı. Yerel sunucuyla test et.

Benzetme kapsayıcı bavul, kodek katlama tekniği gibi: aynı bavula farklı şekilde katlanmış kıyafetler girer.

`<table>` `<tr>` `<td>` `<th>`

Tablo temelleri

tablo.html

```
<table>
  <tr>
    <td> </td>
    <th>Knocky</th>
    <th>Flor</th>
  </tr>
  <tr>
    <th>Breed</th>
    <td>Jack Russell</td>
    <td>Poodle</td>
  </tr>
</table>
```

table: tablonun tamamı. tr: satır. td: veri hücresi.

th: başlık hücresi; kalın ve ortalı, ekran okuyucu ilişkilendirir.

Tablo sadece tablo verisi içindir

Satır ve sütun başlığıyla değer aranan veri: yaş listesi, ders programı, havuz saatleri. Eskiden sayfa düzeni tabloyla yapılırdı; bugün bunun için CSS var.

Düzen (layout) için tablo kullanma, çünkü:

Erişilebilirlik: ekran okuyucu tablo yapısını okur, kafa karışır.

Etiket çorbası: karmaşık, bakımı zor kod.

Duyarlı değil: tablolar içeriğe göre boyutlanır, mobile uymaz.

Benzetme ders programı gibi: “Salı” satırı ile “3. saat” sütununun kesiştiği yerde dersi bulursun.

`colspan="2"` `rowspan="2"`

Hücre birleştirme ve sütun grupları

hayvanlar.html

```
<tr>
  <th colspan="2">Animals</th>
</tr>
<tr>
  <th rowspan="2">Horse</th>
  <td>Mare</td>
</tr>
<tr><td>Stallion</td></tr>
```

colspan sütunları, rowspan satırları birleştirir (birimsiz sayı). Birleşen hücre başkasının yerini kapladığı için o satırlarda daha az td/th yazılır.

<colgroup> ve <col>

sutun.html

```
<colgroup>
  <col span="2" />
  <col class="vurgu" />
</colgroup>
```

Bütün bir sütunu tek seferde stillemek için. table'ın hemen ardından yazılır, soldan sayılır, span kaç sütun kapsadığını söyler. Sadece sınırlı CSS özellikleri çalışır; td/th üzerindeki stil col'u ezer.

Benzetme Excel'de “hücreleri birleştir” gibi: birleşen hücre, yanındakilerin yerini yer.

`<caption>` `scope`

Tablo erişilebilirliği

harcama.html

```
<table>
  <caption>Harcamalarım</caption>
  <thead><tr>
    <th scope="col">Ürün</th>
    <th scope="col">Tutar</th>
  </tr></thead>
  <tbody><tr>
    <th scope="row">Saç kesimi</th>
    <td>30</td>
  </tr></tbody>
  <tfoot>…</tfoot>
</table>
```

`<caption>`
table'ın hemen ardından; tablonun ne olduğunu söyler, kullanıcı okuyup okumamaya karar verir.

`thead/tbody/tfoot`
başlık, gövde, alt bilgi. Bu sırayla yazılır; asıl faydası CSS ve yazdırma kontrolü.

`scope`
th neyin başlığı? col, row, colgroup, rowgroup.

`id + headers`
karmaşık tablolarda her td'yi başlıklarının id'lerine bağla. Çoğu tabloda scope yeter.

Benzetme — görmeyen biri tabloyu hücre hücre dinler: scope, her hücrede “Saç kesimi, Tutar: 30” diye fısıldayan rehberdir.

`<form>` `<input>` `<label>`

Formlar: veri nasıl gönderilir?

form.html

```
<form action="./submit_page" method="get">
  <label for="name">Ad:</label>
  <input type="text" name="name"
         id="name" required />
  <label for="email">E-posta:</label>
  <input type="email" name="email"
         id="email" required />
  <button>Kaydol</button>
</form>
```

adres çubuğu

```
/submit_page?name=Bob&email=bob%40bob.com
```

action: veriyi alacak adres. method: HTTP yöntemi (get veriyi URL'ye ekler).

name=value: her kontrol veriyi çift olarak yollar; name yoksa veri gitmez.

type: text, email, password, number, tel, url, color, radio, checkbox…

label for = input id: ekran okuyucu ne sorulduğunu bilir, yazıya tıklayınca alan seçilir.

<button>: form içinde varsayılan submit. reset kullanma, kazara basılır.

Benzetme	kargo formu gibi: name kutunun etiketi, value içindeki eşya, action kargo şubesinin adresi.

`<select>` `<fieldset>`

Diğer kontroller ve doğrulama

`<textarea>`

form.html

```
<textarea name="yorum"
  rows="5" cols="33">
</textarea>
```

Çok satırlı metin. value ve type yok; içerik etiketler arasında.

`<select>`

form.html

```
<select name="ulasim">
  <option>Seç</option>
  <option value="tren">…
</select>
```

Açılır menü. Seçilen option'ın value'su gider. selected ile ön seçim.

`<fieldset>`

form.html

```
<fieldset>
  <legend>Oda tipi</legend>
  <input type="radio" …>
</fieldset>
```

İlgili kontrolleri gruplar; legend grubun başlığı. disabled hepsini kapatır.

Radio: grupta aynı name, biri seçilir. **Checkbox:** her biri ayrı name, değer yoksa “on” gider.
Doğrulama: required ve type=email tarayıcıda anında uyarır, ama atlatılabilir. Güvenlik için sunucu tarafı doğrulama şart: ikisini birlikte kullan.

Benzetme	kapıdaki güvenlik görevlisi (tarayıcı) ilk kontrolü yapar, ama kasadaki kontrol (sunucu) her zaman yapılmalı.

DevTools › Elements

HTML hata ayıklama

HTML hoşgörülüdür

Tarayıcı bozuk kodu reddetmez, tahmin ederek düzeltip gösterir. Bu yüzden sözdizimi hatası (kod çalışmaz) nadirdir; çoğu sorun mantık hatası gibi davranır: sayfa açılır ama yanlış görünür.

DOM inspector

F12 → Elements (Firefox'ta Inspector). Senin yazdığın kaynağı değil, tarayıcının gerçekte kurduğu DOM'u gösterir. İkisini karşılaştırınca tarayıcının neyi nasıl tamir ettiğini görürsün.

Sık görülen 3 hata

× Kapanmamış öğe

```
hata
<strong>kalın metin…
```

etkisi istenmeyen yerlere taşar

× Yanlış iç içe geçme

```
hata
<strong><em>…</strong></em>
```

tarayıcı bölüp yeniden sarar

× Kapanmamış öznitelik

```
hata
<a href="https://mozilla.org/>
```

link tamamen kaybolur

Benzetme kötü el yazısını okuyan öğretmen gibi: tarayıcı tahmin eder, ama her zaman senin kastettiğini tahmin etmez.

validator.w3.org

W3C doğrulayıcı ile hata avı

1. validator.w3.org → Validate by Direct Input
2. Belgenin TAMAMINI yapıştır (doctype ve head dahil)
3. Check → satır ve sütun numaralı hata listesi
4. Birkaç hata düzelt, tekrar doğrula

Başarı: yeşil bant “No errors or warnings to show”.

Doğrulayıcı mesajı	Aslında demek ki
`Unclosed element strong`	strong hiç kapanmamış
`End tag strong violates nesting rules`	iç içe geçme yanlış
`End of file reached when inside an attribute value`	bir özniteliğin tırnağı eksik
`Unclosed element ul`	yanıltıcı! asıl suçlu tırnağı eksik `<a>`

Domino etkisi: tek bir eksik tırnak bir sürü hata üretir; kökü düzeltince çoğu kaybolur.

Benzetme	yazım denetimi gibi: tek bir eksik parantez yüzünden bütün paragraf kırmızıya boyanır.

Test your skills · Challenges

Pekiştirme: testler ve uygulama görevleri

6 beceri testi

İlgili dersin hemen ardından gelir; kısa görevlerle konuyu ölçer.

- HTML metin temelleri
- İleri düzey HTML metinleri
- Bağlantılar
- Görseller
- Ses ve video
- Formlar ve butonlar

5 uygulama görevi

1	**Mektup işaretleme**	metin öğeleri ve head
2	**Kuş gözlem sitesi**	belge yapısı, semantik bölgeler
3	**Tanıtım sayfası**	resim ve video gömme
4	**Gezegen veri tablosu**	erişilebilir tablo
5	**Geri bildirim formu**	form kontrolleri ve label

Benzetme — ehliyet gibi: kurallar dersi (tutorial) teorik sınav, uygulama görevi (challenge) direksiyon sınavıdır.

`<svg>` `<iframe>`

Ek kılavuzlar: SVG ve gömme teknolojileri

Vektör grafikleri (SVG)

Raster (PNG, JPG) piksel ızgarasıdır, büyütünce bulanıklaşır. Vektör şekilleri matematikle tanımlar, her boyutta keskin kalır. SVG, XML tabanlı vektör dilidir: logo ve ikonlar için ideal.

svg.html

```
<img src="triangle.svg" alt="üçgen" />
<svg width="100" height="100">
  <circle cx="50" cy="50" r="40" />
</svg>
```

img ile eklemek kolay; satır içi svg ise CSS/JS ile şekillere müdahale etmeyi sağlar.

`<iframe>, <embed>, <object>`

iframe, sayfanın içine başka bir web sayfası gömer: YouTube videosu, Google Haritalar gibi.

gomme.html

```
<iframe src="https://…"
  width="560" height="315" sandbox>
</iframe>
```

Güvenlik: sadece HTTPS kaynak kullan, sandbox ile yetkileri kısıtla.

embed / object: eskiden Flash gibi eklentiler içindi; bugün çoğunlukla PDF gömmek için.

Benzetme iframe, duvardaki pencere gibi: dışarıyı (başka siteyi) gösterir, ama sandbox o pencereye parmaklık takar.

</html>

Özet: bugün ne öğrendik?

Sözdizimi

öğe = açılış + içerik + kapanış; doğru iç içe geç, tırnak kullan

head

charset, title, meta, link, script ve lang

Metin

başlık sırası, em/strong, üç liste türü, gelişmiş metin

Yapı ve link

semantik bölgeler, göreli yollar, açıklayıcı link metni

Medya

alt metni, figure, çoklu source, controls, altyazı

Tablo ve form

caption, scope, label for, name=value, çift doğrulama

Benzetme	HTML binanın iskeleti, CSS boyası, JS elektriği: iskelet sağlam değilse boya da elektrik de işe yaramaz.

SORU-CEVAP ZAMANI

Teşekkürler!

Sorularınızı bekliyoruz.

Elit Backend | Structuring content with HTML

Kaynak: MDN Web Docs · Learn web development › Core › Structuring content with HTML

sonuc.html

```
<footer>
  <p>Dinlediğiniz için
    <strong>teşekkürler</strong>
  </p>
  <a href="#sorular">
    Sorular?
  </a>
</footer>
```

Uyarı: Bu içerik, SCÜ Şarkışla UBYO Web Programlama I dersi kapsamında tamamen eğitim amaçlı çevrilmiş ve derlenmiştir. Orijinal dokümantasyon kaynakları (MDN Web Docs, Vue.js, Three.js vb.) kendi orijinal lisanslarına (CC-BY-SA, MIT) tabidir. Bu çalışmanın hiçbir ticari amacı yoktur.